CATT-KV

Causal Attention-Aware Tensor-Train KV-Cache Compression

SVD-free block updates, RoPE-equivariant tensor contraction, sparse innovations, and reconstruction-free attention

CATT-KV asks whether an evolving KV cache can be stored as causally appendable tensor cores and consumed directly by attention, so that compression reduces both capacity and bytes read per decoded token rather than creating a decompression bottleneck.

Positioning. This is a *Technique Paper*, not "Tucker instead of SVD." Its novelty must come from the complete mechanism: dyadic causal tensorization, an SVD-free attention-weighted update, analytic RoPE handling, direct core-domain attention, and a payback rule that declines to compress when runtime savings cannot recover compression cost.

Rigorous research proposal • July 2026

Contents

1 Executive decision **1**

2 Research gap and novelty boundary **1**

3 Thinking template and logic chain **2**

4 Formal problem statement **3**

5 Causal TT representation **3**
5.1 Age-triggered SVD-free update 3

6 Attention-aware fitting and sparse innovations **3**

7 RoPE-equivariant direct TT attention **4**

8 Theory targets **4**

9 System design and cost model **5**

10 Experimental plan **5**
10.1 Research questions 5
10.2 Models, workloads, and baselines 5
10.3 Metrics and ablations 6

11 Reviewer attacks and pre-registered failure cases **6**

12 Paper outline and contribution discipline **6**

13 Execution schedule **7**

14 Final verdict **7**

1 Executive decision

One-sentence key idea

Tensorize each aged KV block along causal position digits, represent it with SVD-free Tensor-Train cores plus attention-critical sparse innovations, and evaluate RoPE attention directly from those cores without materializing dense keys or values.

The project has high upside but a hard empirical gate. Tensor-Train (TT) storage is irrelevant if enumerating logits or updating cores is slower than reading dense or INT4 KV. Accordingly, the first deliverable is not a full model integration; it is a payback study on recorded per-layer KV blocks. Only a positive memory–latency–quality frontier authorizes the full implementation.

Axis	Required contribution	Gate
Representation	Causal TT over token digits, with RoPE-pair structure retained	$\geq$ 4× physical KV reduction before factor quantization
Quality	Attention-output objective and structured sparse correction	$<$ 1 LongBench point or $<$ 2% relative PPL loss
Runtime	Fused logits/value contraction, no dense old-KV reconstruction	$\geq$ 1.3× long-context decode speedup
Amortization	Compress a block once when it ages; admit only positive-payback blocks	$<$ 5% amortized overhead
Baseline strength	Compare at matched physical bytes against low-bit quantization	Beat the INT4 Pareto frontier on at least one system axis

2 Research gap and novelty boundary

The KV cache for a decoder layer is naturally a tensor over KV head, token position, and feature. However, "using a tensor" is not new. DecoQuant uses a matrix-product-operator decomposition to redistribute outliers before quantization [1]; JoLT compresses token and feature axes with partial Tucker plus a rotated low-bit residual [2]; Tucker-KV publicly proposed HOSVD/HOOI refinement and rank allocation [3]. Matrix methods already cover offline low-rank projection (Palu and Eigen Attention) [4, 5], online subspace updates (OjaKV) [6], and RoPE-aligned reconstruction avoidance (RAP) [7].

Method family	Representation	Online update	Dense reconstruction	Gap left for CATT-KV
Palu / Eigen	Matrix projection	No / calibration	Full or fused projection path	No causal multiway model
OjaKV	Adaptive matrix basis	Periodic PCA-style	Reconstructed approximate KV	No tensor cores or direct TT attention
DecoQuant	MPO local tensors	Data-free transform	Dequantization path	Tensorization serves quantization, not causal direct attention
JoLT	Partial Tucker + residual	Block compression	Representation-focused	Uses HOSVD/randomized-SVD family; no causal TT execution path
RAP	RoPE-pair pruning	No	Avoided through aligned absorption	Pruning, not data-adaptive tensor compression
CATT-KV	Causal TT + sparse innovation	SVD-free, block-wise	**Never for old blocks**	Attention-aware, RoPE-equivariant, payback-gated

Claim allowed only if the experiments support it

The publishable novelty is *not* TT alone. It is an attention-executable, causal TT cache: the decomposition algorithm, positional operator, error correction, kernel, and admission rule are designed as one inference mechanism.

3 Thinking template and logic chain

Stage	Proposal content
Research background	Long-context decoding repeatedly reads an expanding KV cache. Quantization, projection, eviction, and recent Tucker methods reduce storage, but runtime wins depend on whether attention can consume the representation without expensive update or reconstruction.
Limitation 1	Tucker/MPO cache methods do not provide a causally appendable SVD-free TT update whose cost is explicitly amortized against future KV reads.
Limitation 2	Generic low-rank representations ignore RoPE's position-dependent pair rotations; naive projection can force reconstruction or inflate rank.
Limitation 3	Frobenius reconstruction loss and byte count do not control softmax routing or end-to-end latency.
Key idea	Store aged causal blocks as RoPE-aware TT cores plus attention-critical sparse innovations and compute attention directly in that domain.
Challenge 1	A growing sequence changes tensor extent, while per-token decomposition would erase the expected latency savings.
Challenge 2	Query-dependent softmax and RoPE must be evaluated without expanding a $B \times d$ block.
Challenge 3	Low tensor error can still change attention routing; rank and residual budgets must optimize quality, bytes, and payback jointly.
Module A	Age-triggered SVD-free TT fitting and a block admission controller.
Module B	RoPE-equivariant direct TT logits and value aggregation kernels.
Module C	Query-sketch-weighted ranks plus structured sparse innovations and error bounds.
Contributions	C1 causal TT cache (Sec. 5); C2 direct RoPE tensor attention (Sec. 7); C3 quality/payback controller and guarantees (Secs. 6–8); C4 reproducible system evaluation (Sec. 10).

All four consistency checks pass: each limitation is answered by the key idea; each challenge is a mechanism-level consequence of implementing it; Modules A–C map one-to-one to the challenges; and every contribution names a method or evaluation section. There are no orphan modules.

4 Formal problem statement

For layer ℓ, KV head h, and a closed block b of B token positions, let

$$K_{\ell hb}, V_{\ell hb} \in \mathbb{R}^{B\times d}, \qquad o(q) = \mathrm{softmax}(qK_{\ell hb}^{\top}/\sqrt{d})V_{\ell hb}.$$

The exact recent window remains in the model's native layout. CATT-KV applies only to a block after it leaves that window. The method must choose a representation $\mathcal{C}_b$, sparse correction E_b, and execution kernel satisfying a physical memory budget M and an update budget T_c:

$$\begin{aligned} \min_{\mathcal{C}_b, E_b} \quad & \mathbb{E}_{q\sim\mathcal{Q}_b}\left[\|o(q) - \widehat{o}(q; \mathcal{C}_b, E_b)\|_2^2\right] + \lambda\,\mathrm{Bytes}(\mathcal{C}_b, E_b) + \mu\,\mathrm{Cost}_{\mathrm{decode}}(\mathcal{C}_b, E_b), \\ \text{s.t.} \quad & \mathrm{Cost}_{\mathrm{compress}}(b) \le T_c, \quad \mathcal{C}_b \text{ is consumed without dense } K_b, V_b. \end{aligned} \tag{1}$$

Here $\mathcal{Q}_b$ is a causal query sketch from prompt and recent-query statistics. Since future queries are unknown, the empirical term is paired with a norm-based robust bound in Section 8.

5 Causal TT representation

Factor $B = \prod_{j=1}^{p} n_j$ (usually $n_j = 2$ or 4) and map token position $s \leftrightarrow (i_1, \ldots, i_p)$. For one head,

$$\mathcal{K}_b[i_1, \ldots, i_p, c] \approx G_1^K[i_1]G_2^K[i_2]\cdots G_p^K[i_p]G_{p+1}^K[c],$$

where $G_j[i_j] \in \mathbb{R}^{r_{j-1}\times r_j}$, $r_0 = r_{p+1} = 1$, and $c \in [d]$. The value cache has separate cores because K and V have different spectra and roles.

Proposition 1 (Physical TT storage)**.** *Ignoring alignment and metadata, one K block requires* $\sum_{j=1}^{p} r_{j-1}n_j r_j + r_p d$ *scalars, rather than* Bd*. The implementation reports actual allocated bytes, including rank tables, sparse indices, padding, and quantization scales.*

Heads are initially decomposed independently to avoid forcing unrelated head bases. A later ablation adds the head as a TT mode only when it reduces physical bytes without raising contraction cost. This prevents a common tensor-paper mistake: assuming every available axis should be jointly factorized.

5.1 Age-triggered SVD-free update

Standard TT-SVD is excluded because it repeatedly invokes matrix SVD. CATT-KV instead uses a warm-started, fixed-sweep TT-ALS update, with QR or regularized least-squares core solves. Online TT-ALS supplies a relevant streaming foundation [8]; randomized streaming sketches offer an alternative when dense MTTKRP-like passes are too expensive [9].

Block lifecycle

Open: append native K/V to the exact hot block. **Close:** freeze the block after B tokens. **Fit once:** run at most J TT-ALS sweeps and select sparse innovations. **Admit or reject:** retain TT only if predicted payback is positive; otherwise store INT4 or native KV. **Consume:** use direct TT attention for every later query.

6 Attention-aware fitting and sparse innovations

Pure TT-ALS minimizes tensor reconstruction. CATT-KV changes the local weighting using a small query sketch $Q_b = [q_1, \ldots, q_m]$ and a robust residual term. For a candidate approximation, define token influence

$$I_s^K = \max_{q\in\mathcal{Q}_b} \frac{|q^{\top}(K_s - \widehat{K}_s)|}{\sqrt{d}} + \rho\left\|K_s - \widehat{K}_s\right\|_2, \tag{2}$$

$$I_s^V = \bar{\alpha}_s\left\|V_s - \widehat{V}_s\right\|_2, \tag{3}$$

where $\bar{\alpha}_s$ is a conservative attention-mass estimate. The residual stores the largest token–RoPE-pair blocks under $I_s^K + I_s^V$, rather than arbitrary scalar entries. This alignment keeps gather operations coalesced and permits exact pair rotations.

Rank selection minimizes an empirical Lagrangian

$$\widehat{L}(\mathbf{r}, k) = L_{\text{attn}}(\mathbf{r}, k) + \lambda M(\mathbf{r}, k) + \mu C_{\text{decode}}(\mathbf{r}, k) + \nu C_{\text{fit}}(\mathbf{r}, k),$$

where k is the residual-block budget. A greedy controller grows the bond or residual budget with the largest measured loss reduction per added byte, subject to a payback constraint. No claim of globally optimal tensor rank allocation is made.

7 RoPE-equivariant direct TT attention

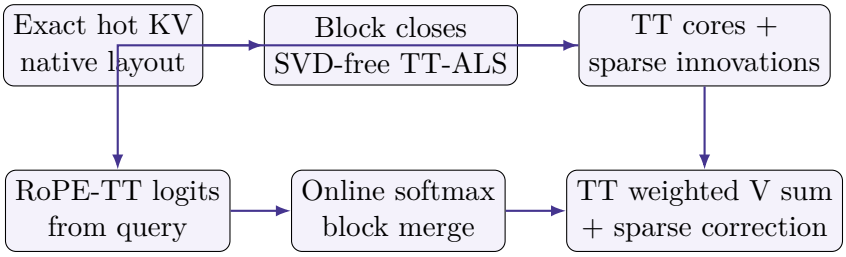
Let $\bar{k}_s$ be the pre-RoPE key and R_s the block-diagonal RoPE matrix. A query at position t uses

$$(R_t q)^\top (R_s \bar{k}_s) = q^\top R_{s-t} \bar{k}_s.$$

For frequency pair j, R_{s-t} is a 2×2 rotation containing $\cos((s-t)\theta_j)$ and $\sin((s-t)\theta_j)$. Under a dyadic encoding of s, the complex sequence $e^{\mathrm{i}s\theta_j}$ factorizes across position bits; its real representation has TT rank at most two for a fixed frequency. CATT-KV therefore constructs the position operator analytically and fuses it with the feature-core contraction instead of rotating or reconstructing all keys.

Lemma 1 (Fixed-frequency RoPE tensorization)**.** *If $s = \sum_{m=0}^{p-1} 2^m i_m$, then $e^{\mathrm{i}s\theta} = \prod_m e^{\mathrm{i}2^m i_m \theta}$. Hence the complex exponential has rank-one dyadic TT form; the coupled real sine/cosine representation has rank at most two.*

The key kernel first contracts the feature/RoPE core with q, then enumerates the B token logits through a prefix-reuse traversal of the position cores. The value kernel contracts the resulting softmax weights with the V position cores and expands only the d-dimensional output. Sparse innovation contributions are accumulated in the same online-softmax pass. Neither kernel allocates a $B \times d$ old-cache tensor.



8 Theory targets

The proposal separates elementary guarantees that can be completed early from ambitious results that are contingent on the final algorithm.

Target theorem 1 (One-query attention-output error)**.**

Let $\epsilon_K = \max_s \left\| K_s - \widehat{K}_s \right\|_2$, $\epsilon_V = \max_s \left\| V_s - \widehat{V}_s \right\|_2$, and $V_{\max} = \max_s \left\| \widehat{V}_s \right\|_2$. Then establish an explicit bound of the form

$$\|o(q) - \widehat{o}(q)\|_2 \le \epsilon_V + 2V_{\max} \frac{\|q\|_2 \, \epsilon_K}{\sqrt{d}},$$

with a tighter version using retained sparse innovations and observed attention mass.

The proof decomposes the error into value approximation and probability-vector perturbation, uses $\|s - \widehat{s}\|_\infty \le \|q\|_2 \, \epsilon_K / \sqrt{d}$, and bounds the softmax Jacobian from ℓ_∞ to ℓ_1. RoPE does not enlarge the key norm because each rotation is orthogonal.

Target theorem 2 (Robust residual selection)**.** *For a query set contained in the span-plus-radius model $q = Q_b a + u$, $\|u\|_2 \le \rho$, show that the influence score upper-bounds the worst-case logit error of each omitted token. Use this to derive a sufficient residual budget for a requested ℓ_∞ logit tolerance.*

Proposition 2 (Payback admission)**.** *Let T_c be one-time block compression time and let $\Delta T = T_{\text{dense/read}} - T_{\text{TT/query}}$. If $\Delta T > 0$, compression improves cumulative time after $N_{\text{pay}} = \lceil T_c / \Delta T \rceil$ future queries. If $\Delta T \le 0$, the runtime must reject TT storage for that block.*

No universal-compressibility theorem is claimed. Recent minimax analysis shows that accurate KV compression requires assumptions on intrinsic cache structure [10]; the paper will state and empirically test those assumptions.

9 System design and cost model

For uniform internal rank r and dyadic $B = 2^p$, approximate per-head storage is $2pr^2 + rd$ scalars per K or V block. A prefix-reuse traversal targets $O(Br^2 + rd)$ key work and $O(Br^2 + rd)$ value work, compared with $O(Bd)$ arithmetic and $O(Bd)$ reads for dense attention. The TT path is plausible only when rank and kernel launch overhead are small enough that reduced memory traffic dominates extra core contractions.

The controller profiles each (ℓ, h, B, r) configuration and stores a lookup table containing physical bytes, compression time, and direct-kernel latency. At block close it predicts remaining queries from the requested maximum generation length and chooses among native, INT4, and TT plus sparse residual. This fallback is part of the correctness of the systems claim, not an ablation.

Implementation stages are:

1. PyTorch reference for TT fitting, direct logits, direct value aggregation, and equality tests.
2. CUDA prototype with fixed $B \in \{128, 256, 512\}$ and ranks $r \in \{4, 8, 12, 16\}$.
3. Fused online-softmax merge across hot, native/INT4, and TT blocks.
4. Hugging Face cache integration for LLaMA/Mistral/Qwen-style GQA and MHA.
5. Jetson Orin measurements plus one desktop/datacenter NVIDIA GPU to avoid device-specific claims.

10 Experimental plan

10.1 Research questions

1. Do per-layer, per-head aged KV blocks have low causal TT ranks before and after RoPE?
2. Does attention-weighted fitting dominate Frobenius TT fitting at matched bytes?
3. Does analytic RoPE contraction reduce rank or latency relative to post-RoPE TT?
4. Do sparse innovations prevent routing failures under retrieval and reasoning workloads?
5. At what context, rank, and device does TT cross its payback point against BF16 and INT4?

10.2 Models, workloads, and baselines

Start with LLaMA-3.2-1B/3B for development, then evaluate at least two architecture families and both GQA and MHA where resources allow. Workloads include PG19 perplexity, RULER retrieval, LongBench, GSM8K or an equivalent reasoning set, and controlled adversarial prompts with isolated

rare facts. Context lengths must include 4K, 8K, 16K, and the longest length supported by the device without extrapolation artifacts.

Baselines are full BF16/FP16, native INT8/INT4 KV, KIVI [12], Palu, Eigen Attention, OjaKV, RAP, DecoQuant, JoLT, and a reconstruction-based TT implementation. Quantization is a mandatory baseline because evidence suggests it can dominate rank reduction at matched budgets [11]. If code for a July 2026 preprint is unavailable, reproduce its reported setting or clearly label a result as paper-reported rather than locally reproduced.

10.3 Metrics and ablations

Report physical allocated bytes, bytes read per decoded token, fit latency, payback queries, time-to-first-token, inter-token latency (median/P95), tokens/s, energy/token, maximum context before OOM, perplexity, exact-match/task accuracy, and attention-output error. Every quality plot uses matched physical bytes; every speed plot includes compression cost.

Required ablations: TT versus TT-SVD (offline quality only); ALS sweep count; token radix/order; pre- versus post-RoPE; query-weighted versus Frobenius objective; residual type and budget; direct versus reconstructed attention; rank controller; factor quantization; and native/INT4 fallback.

11 Reviewer attacks and pre-registered failure cases

Falsification gate

Stop or pivot if, on two model families, (i) median useful TT rank exceeds the direct-kernel break-even rank, (ii) compression payback exceeds 25% of expected remaining generation, or (iii) INT4 matches quality while remaining faster and no larger. A tensor preference is not a reason to continue a dominated method.

Likely reviewer attacks and defenses:

- **"TT is only a different factorization."** Defense: isolate the value of causal update, RoPE operator, direct kernel, and attention-aware residual; do not claim factorization novelty.
- **"The method is slower than quantization."** Defense: matched-byte kernel results and payback admission; report rejection frequency rather than hiding unfavorable layers.
- **"Future queries are unknown."** Defense: robust span-plus-radius bound, multi-query evaluation, and distribution-shift tests.
- **"Tensor ranks are layout artifacts."** Defense: pre-register radix/mode-order search and test ranks across domains and random permutations.
- **"One Jetson is not a top-venue evaluation."** Defense: Jetson demonstrates the target constraint; a second NVIDIA platform establishes portability and separates method from device.

12 Paper outline and contribution discipline

1. **Introduction:** memory traffic, why storage-only compression is insufficient, and the single key idea.
2. **Background/diagnosis:** TT, RoPE, strongest cache baselines, and measured naive-TT failure.
3. **CATT-KV representation:** causal tensorization and SVD-free age-triggered fitting.
4. **Direct attention:** RoPE operator, logits traversal, V aggregation, sparse merge.

5. **Controller and analysis:** attention objective, error bound, memory/operation counts, payback admission.

6. **Evaluation:** quality, physical memory, latency, energy, ablations, failure regions.

7. **Limitations:** rank explosion, short-context overhead, architecture dependence, and custom-kernel maintenance.

The paper should claim three technical contributions and one evaluation contribution at most. It must not simultaneously claim a new benchmark, universal theory, and a new serving system.

13 Execution schedule

Weeks	Deliverable	Exit criterion
1–2	Recorded-KV structural pilot	TT rank/byte curves and direct-contraction microbenchmark
3–5	Reference method	SVD-free fit, sparse residual, numerical equivalence tests
6–9	CUDA direct kernels	No dense allocation; positive break-even region demonstrated
10–12	Model integration	Stable generation, full-cache parity at uncompressed rank
13–16	Evaluation	Two model families, all required baselines and ablations
17–20	Theory and writing	Verified bounds, artifact, complete paper draft

14 Final verdict

Claim allowed only if the experiments support it

If the gates pass, the defensible thesis is: *causal Tensor-Train compression can move old-KV attention from dense memory streaming to bounded-rank core contraction, while RoPE-aware sparse innovations preserve routing and a payback controller prevents negative-speedup cases.*

The idea is an **accept-with-major-revisions research direction**: its logic chain is strong, but novelty and feasibility depend on the direct kernel beating INT4 in a meaningful region. The appropriate venue is NeurIPS/ICLR if the attention objective and guarantees lead; MLSys or ACM ASPLOS if kernel, memory traffic, and end-to-end payback lead; ACM MobiSys if energy/thermal edge deployment becomes the principal result.

References

[1] P. Liu et al. *Unlocking Data-free Low-bit Quantization with Matrix Decomposition for KV Cache Compression.* ACL 2024. https://aclanthology.org/2024.acl-long.133/

[2] R. Krishnan and V. Schulz. *A JoLT for the KV Cache: Near-Lossless KV Cache Compression via Joint Tucker and JL-Residual Allocation for LLMs.* arXiv:2607.12550, 2026. https://arxiv.org/abs/2607.12550

[3] H.-M. Hsu. *Tucker-KV: Provable Tucker Compression of KV Caches with Monotone Refinement and Near-Optimal Budgeting.* OpenReview, 2025–2026. https://openreview.net/forum?id=aQ6RiOijal

[4] C.-C. Chang et al. *Palu: KV-Cache Compression with Low-Rank Projection.* ICLR 2025. https://openreview.net/forum?id=LWMS4pk2vK

[5] U. Saxena et al. *Eigen Attention: Attention in Low-Rank Space for KV Cache Compression.* Findings of EMNLP 2024. https://aclanthology.org/2024.findings-emnlp.899/

[6] Y. Zhu et al. *OjaKV: Context-Aware Online Low-Rank KV Cache Compression.* Findings of ACL 2026. https://aclanthology.org/2026.findings-acl.494/

[7] J. Xin et al. *RAP: KV-Cache Compression via RoPE-Aligned Pruning.* arXiv:2602.02599, 2026. https://arxiv.org/abs/2602.02599

[8] H. Takeda, Y. Miyatake, and D. Furihata. *Online TT-ALS for Streaming Tensor Decomposition with Incremental Orthogonalization.* arXiv:2606.31061, 2026. https://arxiv.org/abs/2606.31061

[9] D. Kressner, B. Vandereycken, and R. Voorhaar. *Streaming Tensor Train Approximation.* SIAM J. Scientific Computing, 2023. https://arxiv.org/abs/2208.02600

[10] L. Haverbeck et al. *The Risk of KV Cache Compression.* arXiv:2607.01520, 2026. https://arxiv.org/abs/2607.01520

[11] S. Salfati. *Quantization Dominates Rank Reduction for KV-Cache Compression.* arXiv:2604.11501, 2026. https://arxiv.org/abs/2604.11501

[12] Z. Liu et al. *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.* ICML 2024. https://arxiv.org/abs/2402.02750